



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών  
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Λειτουργικά Συστήματα  
Εργασία 2(2025-2026)

Ονοματεπώνυμο : Τσάκωνας Βασίλης

Αριθμός Μητρώου : 1115202200245

## Περιεχόμενα

|   |                              |   |
|---|------------------------------|---|
| 1 | Εισαγωγή                     | 1 |
| 2 | Πρώτο μέρος ps               | 2 |
| 3 | Δεύτερο μέρος scheduler MLFQ | 2 |

## 1 Εισαγωγή

Σκοπός της παρούσας εργασίας είναι η μελέτη και η επέκταση του λειτουργικού συστήματος xv6. Στην αρχική του εκδοχή ο scheduler είναι round robin που εκτελεί τις διεργασίες κυκλικά. Εμείς καλούμαστε να φτιάξουμε έναν scheduler που θα υποστηρίξει το μοντέλο MLFQ(Multilevel Feedback Queue).

Έτσι η εργασία χωρίζεται σε 2 μέρη. Το 1ο μέρος αφορά την υλοποίηση της κλήση συστήματος `int getpinfo(struct pstat *)` και την υλοποίηση ενός προγράμματος χρήστη `ps` που θα έχει παρόμοια λειτουργία με την αντίστοιχη εντολή στα Linux. Στο 2ο μέρος καλούμαστε να υλοποιήσουμε τον MLFQ scheduler με 4 ουρές προτεραιότητας, όπου 0 θα είναι αυτή με την υψηλότερη και 3 αυτή με τη χαμηλότερη. Ο scheduler ακολουθεί τους κανόνες που δόθηκαν στην εκφώνηση της εργασίας.

**Σημείωση:** Στον φάκελο kernel βρίσκεται ο κώδικας του kernel space ενώ στον φάκελο user είναι ο αντίστοιχος κώδικας του user space.

## 2 Πρώτο μέρος ps

Για το πρώτο μέρος έγιναν οι εξής τροποποιήσεις στον αρχικό κώδικα του xv6:

- Εισαγωγή νέας δομής "struct pstat" (kernel/pstat.h): Αυτή η δομή είναι ένα στιγμιότυπο των ενεργών διεργασιών του πίνακα proc. Στο struct pstat αποθηκεύονται πληροφορίες για τις ενεργές διεργασίες, όπως pid, ppid, όνομα, μέγεθος, κατάσταση, επίπεδο προτεραιότητας, το πλήθος των ενεργών διεργασιών κλπ) .
- Στα αρχεία kernel/syscall.c, kernel/syscall.h, user/user.h και user/usys.pl προστέθηκε η κλήση συνάρτησης int getpinfo(struct pstat \*).
- Στη αρχείο kernel/proc.h προστέθηκαν οι μεταβλητές priority, ticks\_used, wait\_ticks στη δομή proc, οι οποίες θα χρησιμεύσουν αργότερα στην υλοποίηση του scheduler. Στο αρχείο proc.c στη συνάρτηση allocproc γίνεται και η αρχικοποίηση των μεταβλητών
- Στο kernel/sysproc.c υλοποιείται η συνάρτηση sys\_getpinfo(void) η οποία αντιγράφει τις πληροφορίες από τον kernel στο user space για να μπορεί στη συνέχεια ο χρήστης να έχει πρόσβαση στις πληροφορίες των ενεργών διεργασιών και να υλοποιηθεί η εντολή ps.
- Στο αρχείο user/ps.c υλοποιείται η εντολή ps. Στην πραγματικότητα απλά καλείται η συνάρτηση getpinfo και στη συνέχεια εκτυπώνουμε τις πληροφορίες της δομής.

Ενδεικτική χρήση ps εντολής

```
$ ps
PID   PPID   STATE   SIZE   NAME
1      -1     SLEEP   16384   init
2      1      SLEEP   20480   sh
4      2      SLEEP   20480   sh
5      4      RUNNING 16384   ps
$
```

## 3 Δεύτερο μέρος scheduler MLFQ

Για την υλοποίηση του scheduler χρησιμοποιήθηκε η δομή struct queue mlfq[NQUEUE]. Αυτή η δομή εξυπηρετεί τις 4 ουρές προτεραιότητας. Για τη χρήση των ουρών κατασκευάστηκαν οι εξής συναρτήσεις:

- enqueue\_locked: συνάρτηση που προσθέτει στο τέλος της ουράς τη διεργασία
- dequeue\_locked: συνάρτηση που αφαιρεί τη 1η σε σειρά διεργασία
- enqueue, dequeue: καλούν τις αντίστοιχες locked κλειδώνοντας πρώτα τη δομή mlfq.
- remove\_from\_queue: ψάχνει αν μια διεργασία βρίσκεται στην ουρά και την αφαιρεί. Ο τρόπος που το υλοποιεί αυτό είναι βγάζοντας και στη συνέχεια βάζοντας όλες τις διεργασίες πέρα από αυτή που μας ενδιαφέρει. Ο λόγος που υπάρχει είναι για να μπορούμε στη συνέχεια να αποφύγουμε το starvation και να υπάρχει μηχανισμός αύξησης προτεραιότητας.
- higher\_prio\_runnable: συνάρτηση που ελέγχει αν υπάρχει κάποια διεργασία με υψηλότερο επίπεδο προτεραιότητας σε σχέση με το τρέχον. Αν υπάρχει επιστρέφει 1, αλλιώς 0.

Επιπλέον στο array quantum αποθηκεύουμε το μέγιστο χρονομερίδιο που αναλογεί σε κάθε επίπεδο προτεραιότητας(επίπεδο 0 ->4 timer ticks, επίπεδο 1 ->8 timer ticks , επίπεδο 2-> 16 timer ticks, επίπεδο 3 ->32 timer ticks, κανόνας 6). Οι υπόλοιπες τροποποιήσεις έγιναν στους φακέλους kernel/trap.c και kernel/proc.c.

### Τροποποιήσεις στο kernel/proc.c

Στη συνάρτηση procinit προστέθηκε η αρχικοποίηση των ουρών προτεραιοτήτων. Στην allocproc που είναι η συνάρτηση αρχικοποίησης της κάθε διεργασίας προστέθηκε η αρχικοποίηση των μεταβλητών της proc που αφορούν το MLFQ (priority,ticks\_used,wait\_ticks) και όλες οι διεργασίες πάντα ξεκινούν από το επίπεδο 0(κανόνας 7). Στις συναρτήσεις kfork, kkill τροποποιήθηκε έτσι ώστε όταν η κατάσταση της διεργασίας γίνεται RUNNABLE ταυτόχρονα να εισέρχεται στην αντίστοιχη ουρά(kkill αυτό γίνεται μόνο αν ήταν SLEEPING πριν, kfork εισέρχεται στο επίπεδο 0 καθώς τώρα φτιάχτηκε η διεργασία). Μια επιπλέον αλλαγή που έγινε στην wakeup είναι ότι προστίθεται μια διεργασία που μόλις ξύπνησε στο επίπεδο προτεραιότητας 0. Αυτό γίνεται για να αποφευχθεί το starvation του init καθώς στη περίπτωση όπου έχουμε orphaned zombies η διεργασία Init θα πρέπει να βρίσκεται σε υψηλή προτεραιότητα για να μπορέσει να τρέξει άμεσα και να μη γεμίζει ο πίνακας των διεργασιών. Στη συνάρτηση yield προστέθηκε η εισαγωγή στο αντίστοιχο επίπεδο προτεραιότητας πριν το context switch. Τέλος, η συνάρτηση που τροποποιήθηκε περισσότερο είναι η scheduler(). Αντί για καθαρό round-robin, πλέον χρησιμοποιείται MLFQ: ελέγχει διαδοχικά τις ουρές από επίπεδο 0 έως 3 και εκτελεί την πρώτη διαθέσιμη διεργασία που βρίσκει(κανόνας 4).

### Τροποποιήσεις στο kernel/trap.c

Οι αλλαγές που έγιναν αφορούν τις συναρτήσεις usertrap και kerneltrap. Ουσιαστικά σε κάθε timer tick που περνάει καλείται η νέα συνάρτηση mlfq\_action η οποία κάνει διάφορους ελέγχους που αφορούν το scheduling των διεργασιών(κανόνας 2). Πιο συγκεκριμένα, η mlfq\_action αυξάνει τη μεταβλητή ticks\_used (κανόνας 5) και αν η τρέχον διεργασία έχει ξεπεράσει το συνολικό χρονομερίδιο του επιπέδου που βρίσκεται τότε πέφτει ένα επίπεδο προτεραιότητας(εκτός αν είναι ήδη στο 3) και καλείται η συνάρτηση yield μέσω της οποίας επιτυγχάνεται το context switch(κανόνας 8, 9). Επιπλέον, τσεκάρει και τις υπόλοιπες διεργασίες που είναι σε κατάσταση RUNNABLE, αυξάνει τη μεταβλητή wait\_ticks και κάνει έλεγχο για πιθανό starvation, δηλαδή αν μία διεργασία περιμένει 10 φορές το χρονομερίδιο της στο τρέχον επίπεδο προτεραιότητας, αυξάνεται το επίπεδο προτεραιότητας της(εκτός αν είναι ήδη στο επίπεδο 0), κανόνας 11. Επιπλέον σε κάθε timer tick ελέγχουμε αν υπάρχει κάποια διεργασία με υψηλότερη προτεραιότητα σε σχέση με την τρέχον. Αν αυτό συμβαίνει τότε κάνουμε context switch χωρίς όμως να μηδενίζουμε τη μεταβλητή ticks\_used έτσι ώστε όταν χρονοπρογραμματιστεί η διεργασία ξανά να χρησιμοποιήσει το υπόλοιπο χρονομερίδιο που της αναλογεί(κανόνας 3,10).

Με τη καινούρια υλοποίηση όλα τα τεστ περνάνε με OK.

```
test bigdir: OK
test manywrites: OK
test badwrite: OK
test execout: OK
test diskfull: balloc: out of blocks
balloc: out of blocks
OK
test outofinodes: ialloc: no inodes
OK
ALL TESTS PASSED
$
```